「河边共指星为客，花里空瞻月是卿」

Wi. Zhimu Yang

Hydrangea Academy of Magic, HAM

Magnolia Institute of Science, MIS.HAM

计算理论导引（2）

> *Type:*
>
> *~~Tutorial~~ / Note*
>
> *For:*
>
> *Amateurs / ~~Professionals~~*
>
> *Reference:*
>
> *[1] Michael Sipser . 《Introduction to the Theory of Computation》. 机械工业出版社, 2015*
>
> *Preface:*
>
> *仅作学习交流与分享用途。*
>
> *作为导引性的系列笔记，我们还是只做基础的严谨理论整理和入门级别的介绍，事实上该领域还有十分广袤的内容，从证明到应用我们现在不能一一复述，如果感兴趣，可以查阅相关资料，关键词：形式语言与自动机*

计算理论导引（2）
- 一、Automata(自动机) & Languages(语言)
  - 2.Context-Free Languages(上下文无关语言)
    - Context-Free Grammar(上下文无关文法)
      - Definition of Context-Free Grammar(上下文无关文法的定义)
      - Ambiguity(歧义性)
      - Chomsky Normal Form(Chomsky范式)

Pushdown Automata(下推自动机)
Definition of Pushdown Automata(下推自动机的定义)
Computation of Pushdown Automata(下推自动机的计算)
Equivalence of Pushdown Automata and Context-Free Grammars(下推自动机与上下文无关文法的等价性)
Non-Context-Free Languages(非上下文无关语言)
The Pumping Lemma for Context-Free Languages(上下文无关语言的泵引理)
Nondeterminism(不确定性)
Definition of Deterministic Pushdown Automata(确定型下推自动机的定义)

一、Automata(自动机) & Languages(语言)

2.Context-Free Languages(上下文无关语言)

上一文中我们介绍了Regular Languages(正则语言)和它对应的机器有穷自动机，接下来我们将介绍**Context-Free Grammar(上下文无关文法)**和它的集合**Context-Free Languages(上下文无关语言,CFL)**，以及它对应的机器**Pusdown Automata(下推自动机)**。上下文无关文法是一种更强大的描述语言的数学模型。

Context-Free Grammar(上下文无关文法)

Definition of Context-Free Grammar(上下文无关文法的定义)

> *Definition*
>
> ***Context-Free Grammar(上下文无关文法)***
>
> *上下文无关文法可以描述为一个4元组*(V, Σ, R, S)
>
> *V：Variables(变元集)，有穷集合*
>
> *Σ：Terminals(终结符集)，与V不相交的有穷集合*
>
> *R：Rules(规则集)，有穷集合，每条规则由一个变元和一个由变元及终结符组成的字符串构成*
>
> *S：Start Variable(起始变元)，*$S \in V$
>
> *设u，v和w是由变元及终结符构成的字符串，*$A \to w$*是该文法G的一条规则：*

> *1.称uAv yield(生成) uwv，记作$uAv \Rightarrow uwv$*
>
> *2.若$u = v$，或存在序列$u_1, u_2, \cdots, u_k$，使得*
> $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \cdots \Rightarrow u_k \Rightarrow v(k \geq 0)$，*则称u derive(派生) v，记作$u \overset{*}{\Rightarrow} v$*
>
> *3.该文法G的语言为$\{w \in \Sigma^* | S \overset{*}{\Rightarrow} w\}$*

一个文法由一组Substitution Rule(替换规则)/Production Rule(产生式规则)组成。每个规则包括一个符号和一个字符串，符号称为Variable(变元)，字符串则由变元和另一种称为Terminal(终结符)的符号组成。

根据文法我们能够生成其所描述语言的每一个字符串。生成文法对应序列的过程称为Derivation(派生)，我们可以使用Parse Tree(语法分析树)来描绘派生过程。派生得到的所有字符串构成该文法的语言，记为$L(G)$。

Ambiguity(歧义性)

有时一个文法中能够用多种不同方式产生同一字符串，并有多棵不同的语法分析树，对应多种含义，这显然是我们不期望的。

一个文法歧义地产生一个字符串是指，该字符串有两棵不同的语法分析树，而不是两种不同的派生。两种派生可能仅仅是次序问题而不是结构不同。

故我们定义一个固定次序。

> *Definition*
>
> ***Leftmost Derivation(最左派生)***
>
> *对于文法G的一个字符串w的派生，每一步都是替换最左边剩下的变元。*

> *Definition*
>
> *如果字符串w在上下文无关文法G中有两个或以上的最左派生，则称G ambiguously(歧义地)产生字符串w，如果文法G歧义地产生某个字符串，则称文法G是ambiguous(歧义的)。*

有时对于一个歧义文法能够找到一个产生相同语言的非歧义文法。但是某些上下文无关语言只能由歧义文法产生，我们称为Inherently Ambiguous(固有歧义的)。

Chomsky Normal Form(Chomsky范式)

Chomsky范式是上下文无关文法的最简单有效的形式，这会方便我们使用。

> *Definition*
>
> ***Chomsky Normal Form(Chomsky范式)***
>
> *若一个上下文无关文法的每一个规则满足：*
>
> $$A \to BC$$
>
> $$A \to a$$
>
> *其中，a是任意终结符，A、B、C是任意变元，且B、C不为起始变元，并允许规则$S \to \epsilon$，S为起始变元。*
>
> *则称该上下文无关文法为Chomsky范式。*

> *Theorem*
>
> *Any context-free language is generated by a context-free grammar in Chomsky normal form.*
>
> *任一上下文无关语言都可以用一个Chomsky范式的上下文无关文法产生。*

Pushdown Automata(下推自动机)

下推自动机是一种更为强大的计算模型，在有穷自动机的基础上添加了Stack(栈)，提供了附加存储量，从而能够识别更多语言。栈中存在一个将符号压入栈的过程，称为下推，这也是下推自动机名字的由来。对于栈来说，所有操作都只能在栈顶进行，请谨记。

Definition of Pushdown Automata(下推自动机的定义)

> *Definition*
>
> ***Pushdown Automata(下推自动机)***
>
> *下推自动机可以描述为一个6元组*$(Q, \Sigma, \Gamma, \delta, q_0, F)$
>
> *Q：State(状态集)，有穷集合，所有状态集合*

> Σ：*Input Alphabet(输入字母表)，有穷集合，所有输入允许的字符集合*
>
> Γ：*Stack Alphabet(栈字母表)，有穷集合，所有栈允许的字符集合*
>
> δ：$Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathrm{P}(Q \times \Gamma_\epsilon)$，*Transition Function(转移函数)*
>
> $$\Sigma_\epsilon = \Sigma \bigcup \{\epsilon\}$$
>
> $$\Gamma_\epsilon = \Gamma \bigcup \{\epsilon\}$$
>
> q_0：*Start State(起始状态)*，$q_0 \in Q$
>
> F：*Accept States(接受/终止状态集)*，$F \subseteq Q$

Computation of Pushdown Automata(下推自动机的计算)

> *Definition*
>
> *下推自动机* $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$，*字符串*$w = w_1 w_2 w_3 \cdots w_n$*且其中每一个* $w_i \in \Sigma_\epsilon$。*若存在* Q *中的状态序列* $r_0, r_1, r_2, \cdots, r_n$ *和* Γ^* *中的字符串序列* $s_0, s_1, \cdots, s_m$*满足：*
>
> 1. $r_0 = q_0$*且*$s_0 = \epsilon$，*从起始状态和空栈开始*
> 2. $(r_{i+1}, b) \in \delta(r_i, w_{i+1}, a)(i = 0, 1, \cdots, i-1)$，*其中* $s_i = at, s_{i+1} = bt,\ a, b \in \Gamma_\epsilon, t \in \Gamma^*$，*机器M的状态改变完全由当前状态、栈顶符号、下一输入符号决定*
> 3. $r_n \in F$，*机器处在终止状态*
>
> *则字符串*s_i*是机器M在计算过程中接受分支的栈内容序列。*

Equivalence of Pushdown Automata and Context-Free Grammars(下推自动机与上下文无关文法的等价性)

上下文无关文法与下推自动机能力上是等价的，均可描述上下文无关语言类。

> *Theorem*
>
> *A language is context free if and only if some pushdown automata recognizes it.*
>
> *一个语言是上下文无关的，当且仅当存在下推自动机能够识别它。*
>
> - *Lemma*

> *If a language is context free,then some pushdown automata recognizes it.*
>
> *若一个语言上下文无关，则存在下推自动机能够识别它。*
>
> - *Lemma*
>
> *If a pushdown automata recognizes some language,then it is context free.*
>
> *若一个语言能够被下推自动机识别，则它是上下文无关的。*

> *Corollary*
>
> *Every regular language is context free.*
>
> *每个正则语言都是上下文无关的。*

Non-Context-Free Languages(非上下文无关语言)

同样的，我们可以通过上下文无关语言的泵引理来证明某些语言是非上下文无关的。

The Pumping Lemma for Context-Free Languages(上下文无关语言的泵引理)

> *Theorem*
>
> ***Pumping Lemma for Context-free Languages(上下文无关语言的泵引理)***
>
> *If A is a context-free languages,then there is a number p(the pumping length) where,if s is any string in A of length at least p,then s may be divided into five pieces $s = uvxyz$ satisfying the conditions:*
>
> *1.for each $i \geq 0$ $uv^ixy^iz \in A$*
>
> *2.$|vy| > 0$*
>
> *3.$|vxy| \leq p$*
>
> *若A是上下文无关语言，则存在一个数p(泵长度)，使得A中任何一个长度不小于p的字符串s都能被划分为5段$s = uvxyz$且满足：*
>
> *1.对于每个$i \geq 0$ $uv^ixy^iz \in A$*
>
> *2.$|vy| > 0$*
>
> *3.$|vxy| \leq p$*

Nondeterminism(不确定性)

确定型有穷自动机与非确定型有穷自动机具有等价的语言识别能力，那么下推自动机如何？

Definition of Deterministic Pushdown Automata(确定型下推自动机的定义)

> *Definition*
>
> ***Deterministic Pushdown Automata(确定型下推自动机)***
>
> *确定型下推自动机可以描述为一个6元组*$(Q, \Sigma, \Gamma, \delta, q_0, F)$
>
> Q：*State(状态集)，有穷集合，所有状态集合*
>
> Σ：*Input Alphabet(输入字母表)，有穷集合，所有输入允许的字符集合*
>
> Γ：*Stack Alphabet(栈字母表)，有穷集合，所有栈允许的字符集合*
>
> δ：$Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to (Q \times \Gamma_\epsilon) \bigcup \{\emptyset\}$，*Transition Function(转移函数)*
>
> $\Sigma_\epsilon = \Sigma \bigcup \{\epsilon\}$
>
> $\Gamma_\epsilon = \Gamma \bigcup \{\epsilon\}$
>
> δ*满足：*
>
> *给定任意一组*$q \in Q, a \in \Sigma, x \in \Gamma$，*在* $\delta(q, a, x), \delta(q, a, \epsilon), \delta(q, \epsilon, x), \delta(q, \epsilon, \epsilon)$ *中有且仅有一个不是*$\emptyset$
>
> *这避免了在相同的情况下执行不同的动作*
>
> q_0：*Start State(起始状态)*，$q_0 \in Q$
>
> F：*Accept States(接受/终止状态集)*，$F \subseteq Q$

与有穷自动机相反的，非确定型下推自动机相比确定型下推自动机具有更强的语言识别能力。